

FREELANCER SB WORKS

Project Description:

The Freelancer SB Works is a MERN stack web application that connects clients and freelancers in a single platform. Clients can post projects, review proposals, and hire suitable freelancers, while freelancers can browse jobs, submit bids, and work on assigned projects. The system provides secure authentication, profile management, and smooth communication, making it easy for both clients and freelancers to collaborate efficiently.

Scenario:

A client visits the platform to get a task completed, such as developing a website or designing a logo. The client registers, logs in, and posts a project with details like budget, deadline, and requirements. Freelancers on the platform browse available projects and submit their proposals with pricing and timelines.

The client reviews multiple proposals, checks freelancer profiles, and selects the most suitable candidate. Once hired, the freelancer starts working on the project and communicates with the client through the platform. After successful completion, the client approves the work and provides payment and feedback, completing the workflow.

System Requirements:

1. Software Requirements:

These are the essential tools and platforms required to develop, test, and run the application efficiently.

Operating System: Windows 10/11, macOS, or Linux — Supports cross-platform development and testing.

Node.js (v16 or above): Provides the runtime environment for building and managing frontend logic and Powers the server-side logic and handles API routing. 🖱️ [Download Node.js](#)

npm (v8 or above): A package manager required to install dependencies for React and related libraries.

React.js: A JavaScript library for building dynamic and responsive user interfaces.

Browser: Google Chrome / Firefox (latest version) — For rendering and testing the UI in real-time.

Express.js: A lightweight web framework for building RESTful APIs.

MongoDB: A NoSQL database used to store structured and unstructured data related to farms, users, and resources. 🖱️ [Download MongoDB](#)

Postman: Tool for testing APIs during development.

Visual Studio Code: Preferred code editor with built-in Git and terminal support.

Git & GitHub: For version control and collaborative development.

2. Hardware Requirements:

Describes the minimum and recommended specifications needed to support the development and usage of the application.

Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — Ensures fast compilation and multitasking during development.

RAM: Minimum 8 GB (16 GB recommended) — For handling development servers, IDEs, and browser testing simultaneously.

Storage: At least 1 GB free space — Required for package installations, MongoDB setup, and local project files.

Display: 1366x768 or higher — Recommended for optimal coding experience and application layout visualization.

Project Architecture:

- **Technical Architecture:**

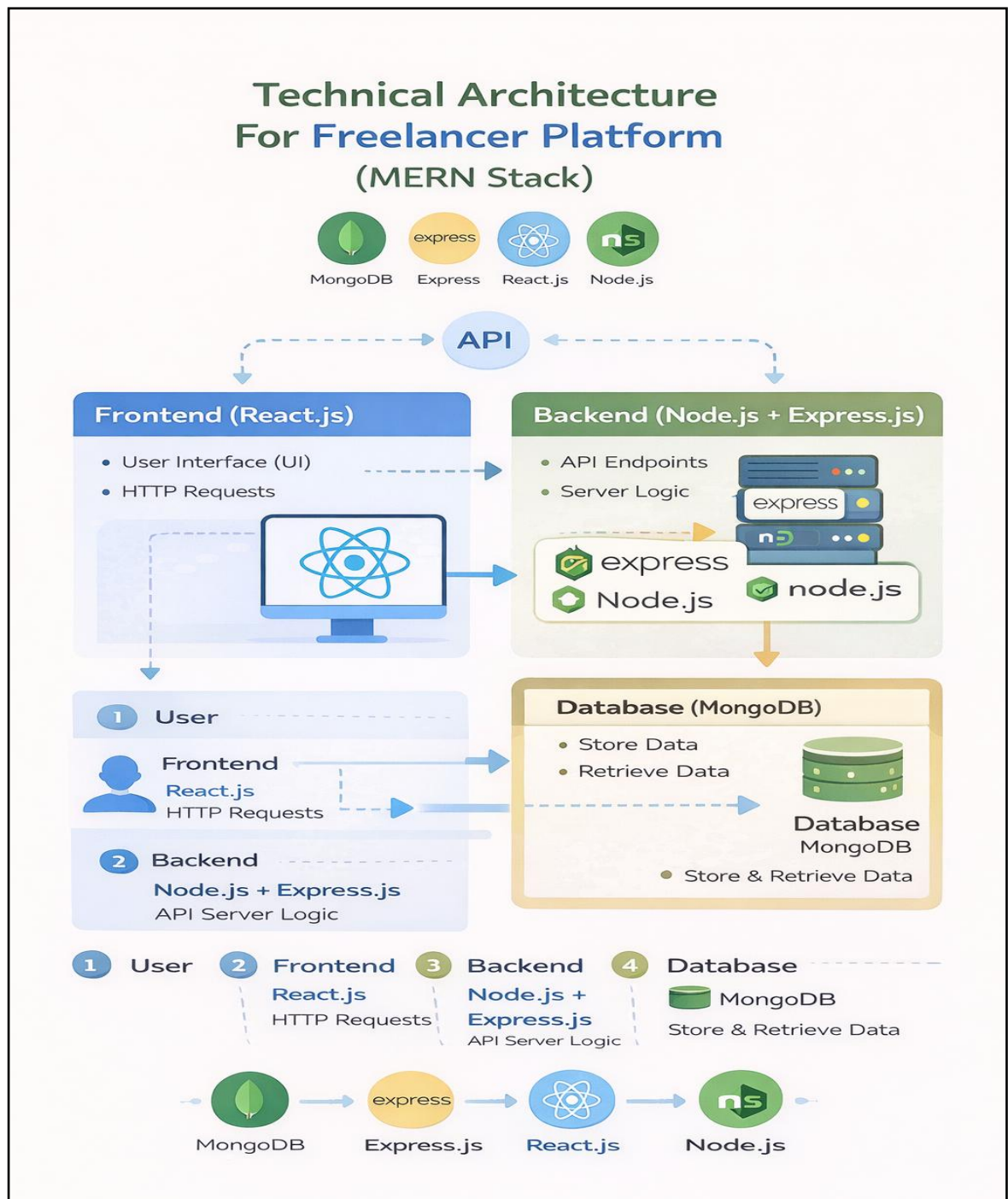


Figure Technical Architecture

1. User (Client / Freelancer)

- The process starts when a user interacts with the application
- Example: login, post a job, apply for a job

The user does everything through the frontend (website UI)

2. Frontend (React.js)

- Built using **React.js**
- Shows UI like dashboard, job list, profile, etc.
- Sends requests to backend using HTTP (API calls)

Example: User clicks “Post Job” → React sends data to backend

👉 React only handles **UI and user interaction**

3. Backend (Node.js + Express.js)

- Backend receives requests from frontend
- Built using **Node.js + Express.js**
- Contains all logic (authentication, job handling, bidding)

What it does:

- Validates user data
- Processes requests
- Decides what data to store or send

Backend is the **brain of the application**

4. Database (MongoDB)

- Stores all data:
 - Users
 - Jobs
 - Proposals
 - Reviews

What happens:

- Backend stores data in MongoDB
- Backend retrieves data when needed

MongoDB is the **data storage**

• ER Diagram:

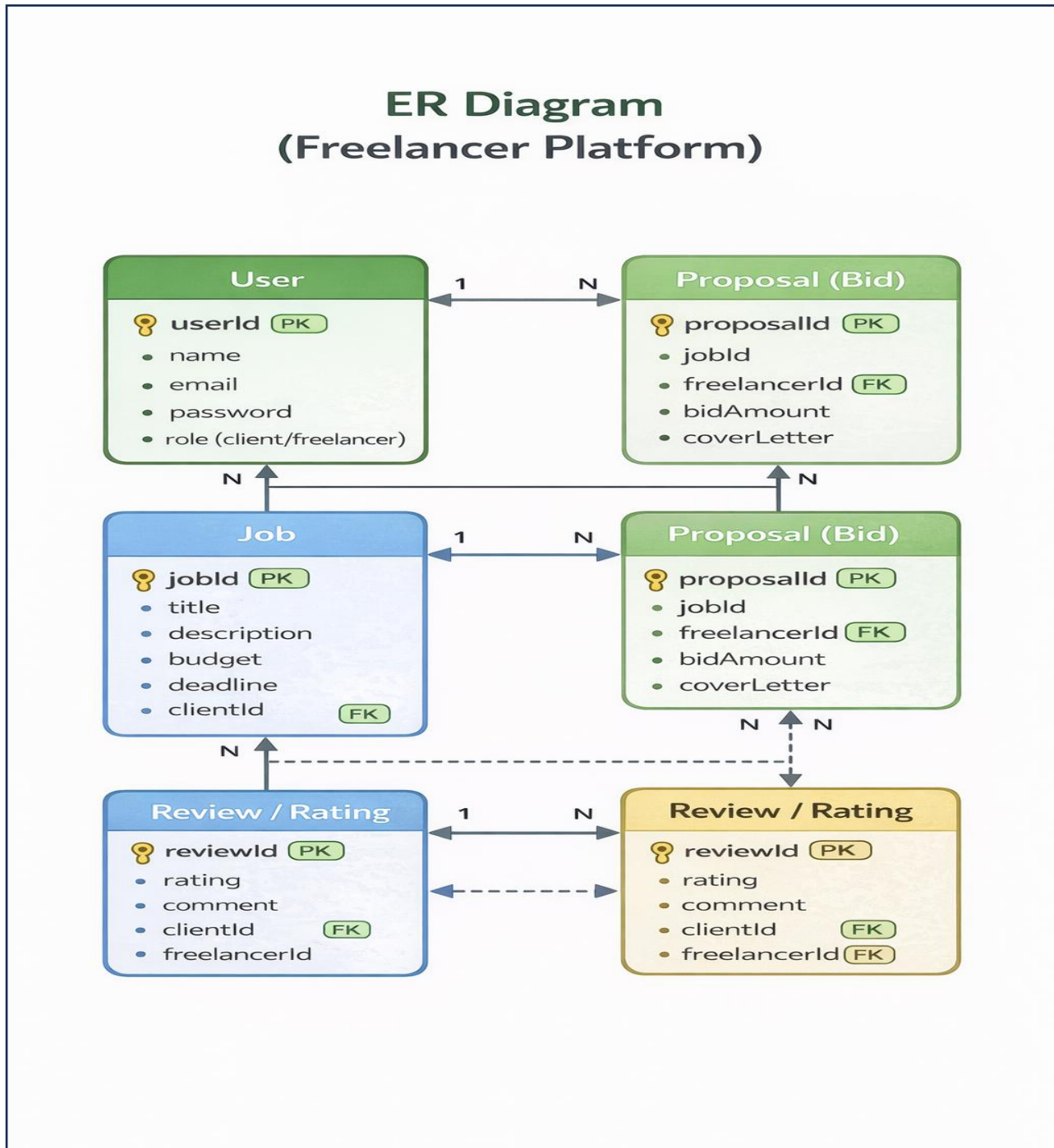


Figure ER Diagram

1. User

- Attributes: userId, name, email, password, role (client/freelancer)
- A user can be either a client or a freelancer

This is the main entity of the system

2. Job

- Attributes: jobId, title, description, budget, deadline, clientId
- Created by client

One client can post multiple jobs

3. Proposal (Bid)

- Attributes: proposalId, jobId, freelancerId, bidAmount, coverLetter
- Sent by freelancers to apply for jobs

Connects Freelancer ↔ Job

4. Review / Rating

- Attributes: reviewId, rating, comment, clientId, freelancerId
- Given after job completion

👉 Helps build trust in platform

Relationships (Very Important)

1. User → Job

- One-to-Many (1:N)
- One client can post many jobs

User (Client) → Job

2. User → Proposal

- One-to-Many (1:N)
- One freelancer can send many proposals

User (Freelancer) → Proposal

3. Job → Proposal

- One-to-Many (1:N)
- One job can have many proposals

Job → Proposal

4. User ↔ User (Review)

- Many-to-Many (M:N) via Review
- Clients review freelancers

User → Review → User

- **Features:**

Features of Freelancer Platform

1. User Authentication

- Register & Login system
- Role-based access (Client / Freelancer)
- Secure password using encryption (bcrypt, JWT)

2. User Profiles

- Create and update profile
- Add skills, experience, and details
- Separate dashboards for client & freelancer

3. Job Posting (Client)

- Clients can post jobs
- Add title, description, budget, deadline
- Manage posted jobs

4. Job Browsing (Freelancer)

- Freelancers can view available jobs
- Filter/search jobs
- Apply to jobs

5. Proposal / Bidding System

- Freelancers submit proposals (bids)
- Include bid amount & cover letter

- Clients can view all proposals

6. Hiring System

- Client selects a freelancer
- Job gets assigned
- Status changes (open → in progress → completed)

7. Review & Rating

- Clients can rate freelancers
- Feedback system for trust building

8. Dashboard

- Personalized dashboard for both users
- Track jobs, proposals, and activity

9. REST API Integration

- Frontend communicates with backend via APIs
- Handles all CRUD operations

10. Secure Backend

- Middleware for authentication
- Error handling & validation

• Roles and Responsibilities:

Roles in the System

1. Client

- Person who posts jobs and hires freelancers

2. Freelancer

- Person who applies for jobs and completes work

Responsibilities

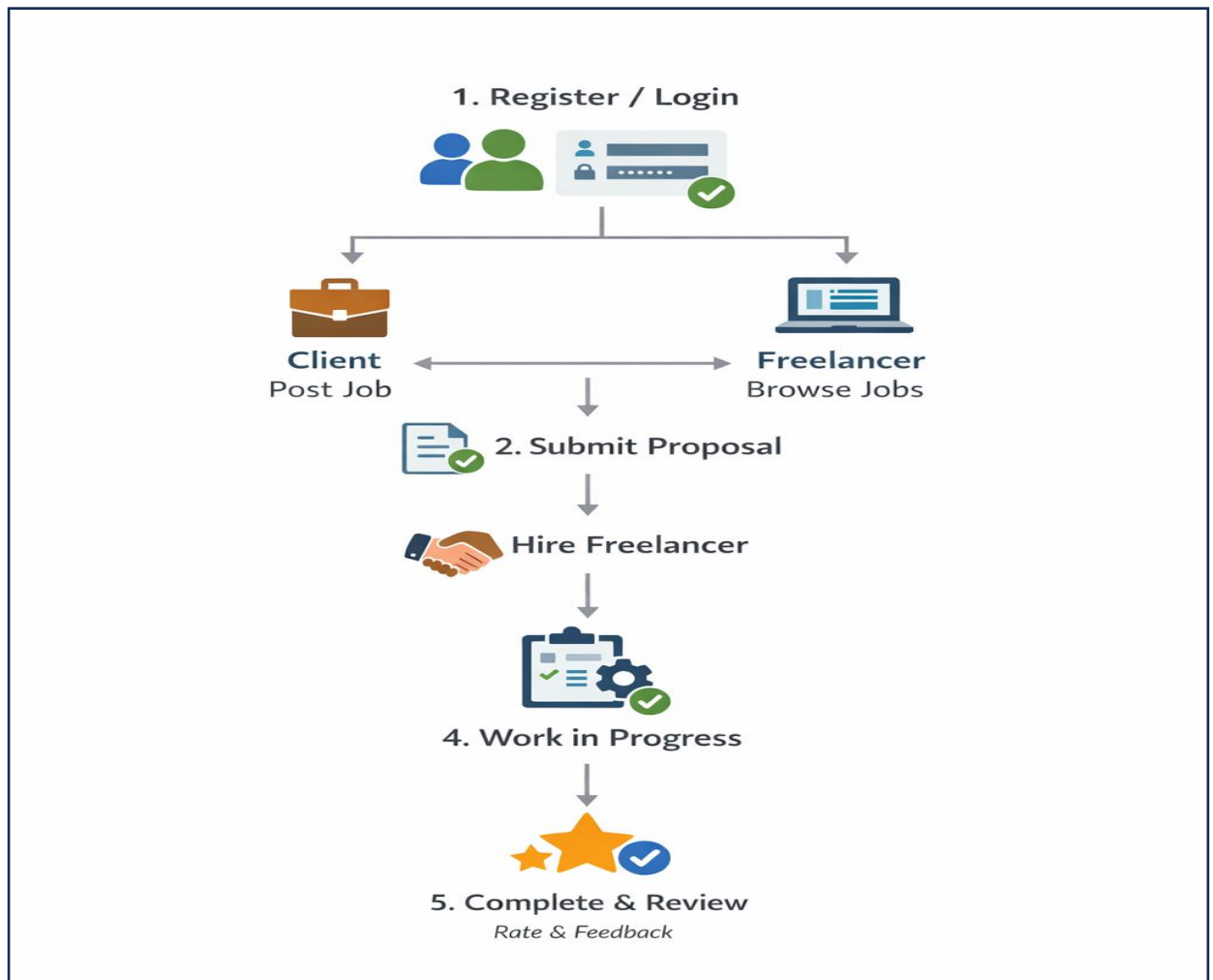
Client Responsibilities:

- Register/Login into the platform
- Create and manage profile
- Post new jobs with details (budget, deadline, description)
- View freelancers' proposals
- Select and hire a freelancer
- Monitor job progress
- Give ratings and reviews after completion

Freelancer Responsibilities:

- Register/Login into the platform
- Create and update profile (skills, experience)
- Browse available jobs
- Submit proposals (bid amount + cover letter)
- Communicate with clients (if implemented)
- Complete assigned work within deadline
- Receive ratings and feedback

- **User Flow:**



Step-by-Step User Flow

1. User Registration / Login

- User signs up or logs in
- Chooses role: Client or Freelancer

2. Dashboard Access

- User is redirected to dashboard
- Based on role:
 - Client Dashboard
 - Freelancer Dashboard

3. Client Flow

- Post a job (title, budget, deadline)

- View proposals from freelancers
- Select/hire a freelancer

4. Freelancer Flow

- Browse available jobs
- Apply to jobs (submit proposal)
- Wait for client selection

5. Job Execution

- Freelancer works on assigned job
- Client monitors progress

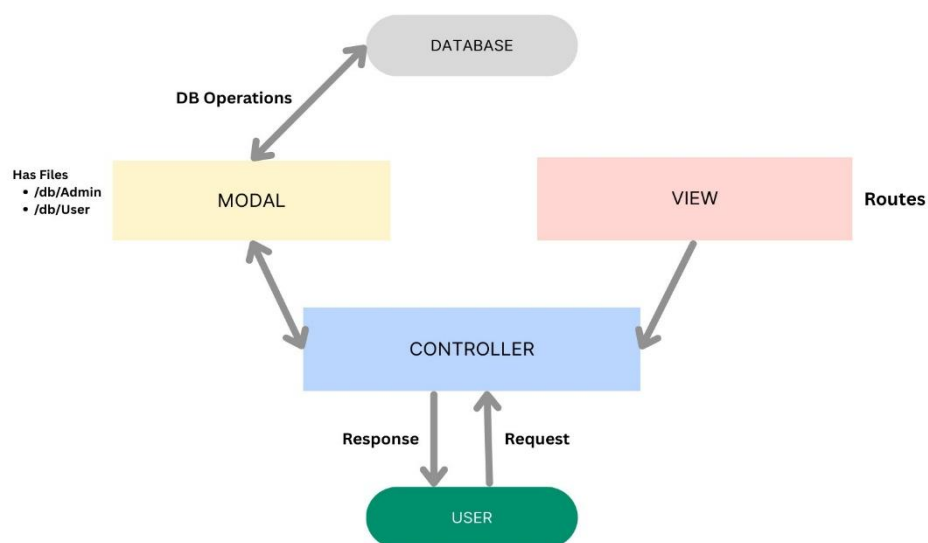
6. Completion

- Job marked as completed
- Payment process (if implemented)

7. Review & Rating

- Client gives rating & feedback
- Freelancer profile gets updated

• MVC Pattern:



The Agri-Tech backend application follows the Model-View-Controller (MVC) architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.

Model Layer (Data Layer):

The **Model** layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database using those schemas. The models are implemented using **Mongoose**, which provides a schema-based solution to model application data for MongoDB.

Controller Layer:

The Controller layer acts as an intermediary between the view (routes) and the model. It receives incoming requests, processes the input (which may include validation or transformation), calls the appropriate methods from the model, and then returns a response to the client.

View Layer (Routing Layer):

In the context of a backend REST API, the View is implemented as the routing layer, where various endpoints are defined. These endpoints determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions.

Advantages of Using MVC in This Project:

- Separation of Concerns: Each layer has a clearly defined responsibility, improving readability and maintainability.
- Scalability: New features can be added easily by creating new routes, controllers, and models.
- Reusability: Logic in controllers and models can be reused across

multiple parts of the application.

- Testing: Each layer can be tested independently, especially the controllers and models.
- Collaboration-Friendly: Multiple developers can work simultaneously on different layers without conflict.

Project Setup and Configuration:

- **Creating Project Folder**

1. Create a new folder with your .
2. Inside that folder create two new folders.
3. Name one as ****Client****.
4. Name another one as ****Server****.
5. Now open that folder in VS Code.

- **Client Setup (installing React App)**

- Open the Client folder in the terminal of VScode.
 - `npm create vite@latest . -- --template react`
- Select React framework from the given options.
 - Select a framework:

| React
- Select JavaScript variant from the given options.
 - Select a variant:

| JavaScript
- Now lets navigate to the client folder by giving the following command.
 - `cd client`
- To install all the packages run the following command.
 - `npm install`

- To start the React server type the following command.
 - `npm run dev`

- **Server Setup (npm init)**

- Open Server folder in terminal of VScode.
 - `npm init -y`
- Create files:
 - `server.js`
- Create folders:
 - `models`
 - `controllers`
 - `routes`

Backend Development:

Reference video with voiceover or screen shots for backend code

Technology:

Core Tech

- **MongoDB**
Database used to store users, projects, bids, messages
- **Node JS**
Handles API routes (`authRoutes.js`, `projectRoutes.js`, etc.)
- **Express JS**
Runs your backend server (`index.js`)

Additional Backend Tech

Mongoose

→ Used in models/Schema.js to define database schemas

JSON Web Token(JWT)

→ Used for authentication (login/register system)

bcrypt

→ Encrypts user passwords securely

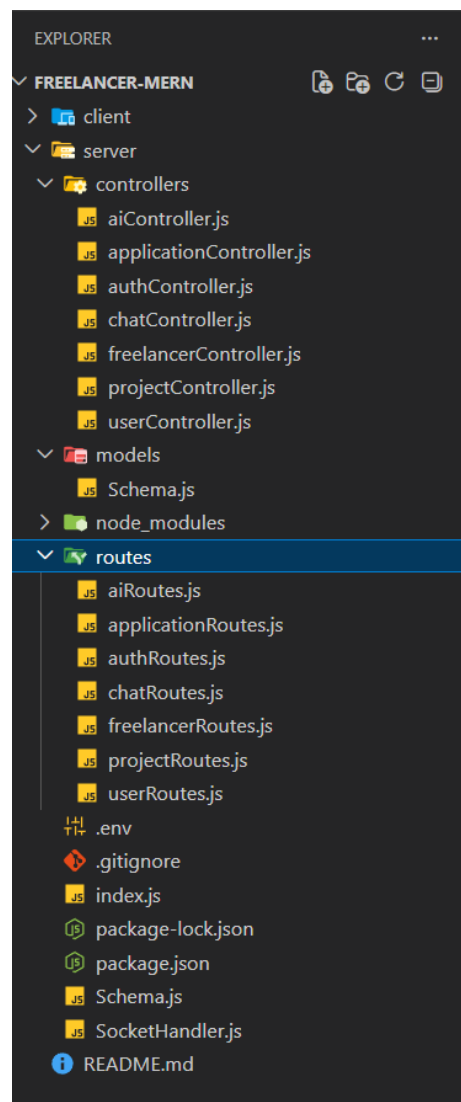
Socket.io

→ Used in SocketHandler.js for chat system

dotenv

→ Stores secrets in .env file

Backend Folder:



Architecture Type

Your project follows:

MVC (Model-View-Controller) Architecture

- **Models** → models/Schema.js
- **Controllers** → controllers/*.js
- **Routes** → routes/*.js
- **Server Entry** → index.js

Figure of Backend

Index.js

1. Server and Routes Setup

```
server >  index.js > ...  
1  import dotenv from 'dotenv';  
2  dotenv.config();  
3  
4  import express from 'express';  
5  import cors from 'cors';  
6  import bodyParser from 'body-parser';  
7  import mongoose from "mongoose";  
8  
9  import authRoutes from './routes/authRoutes.js';  
10 import freelancerRoutes from './routes/freelancerRoutes.js';  
11 import projectRoutes from './routes/projectRoutes.js';  
12 import applicationRoutes from './routes/applicationRoutes.js';  
13 import userRoutes from './routes/userRoutes.js';  
14 import chatRoutes from './routes/chatRoutes.js';  
15 import aiRoutes from './routes/aiRoutes.js';
```

2. CORS Configuration


```
// ✅ CORS must come FIRST, before any other middleware
app.use(cors({
  origin: (origin, callback) => {
    if (!origin || allowedOrigins.includes(origin)) {
      callback(null, true);
    } else {
      callback(new Error('Not allowed by CORS'));
    }
  },
  credentials: true
}));
```

3. Database Connection

```
const PORT = process.env.PORT || 6001;

mongoose.connect(process.env.MONGODB_URI)
  .then(() => {
    console.log("MongoDB Connected");
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
    });
  })
  .catch((err) => console.log("DB connection error:", err));
```

Schema.js

1. User Schema

```
const userSchema = mongoose.Schema({
  username: {
    type: String,
    require: true
  },
  email: {
    type: String,
    require: true,
    unique: true
  },
  password: {
    type: String,
    require: true
  },
  usertype:{
    type: String,
    require: true
  }
})
```

2. Freelancer Schema

```
const freelancerSchema = mongoose.Schema({
  userId: String,
  skills: { type: Array, default: [] },

  description: { type: String, default: "" },

  currentProjects: { type: Array, default: [] },

  completedProjects: { type: Array, default: [] },

  applications: { type: Array, default: [] },

  funds: { type: Number, default: 0 },
})
```

authControllers.js

1. Register Controller

```
export const registerUserController = async (req, res) => {
  try {
    const { username, email, password, usertype } = req.body;

    const existingUser = await User.findOne({ email });
    if (existingUser) {
      return res.status(400).json({ msg: "User already exists" });
    }

    const salt = await bcrypt.genSalt(10);
    const hashPassword = await bcrypt.hash(password, salt);

    const user = new User({ username, email, password: hashPassword, usertype });
    await user.save();

    res.status(201).json({
      _id: user._id,
      username: user.username,
      email: user.email,
      usertype: user.usertype,
      token: user._id.toString(),
    });
  } catch (error) {
    res.status(500).json({ msg: error.message });
  }
}
```

2. Login Controller

```
export const loginUserController = async (req, res) => {
  try {
    const { email, password } = req.body;

    const user = await User.findOne({ email });
    if (!user) {
      return res.status(404).json({ msg: "User does not exist" });
    }

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      return res.status(401).json({ msg: "Invalid credentials" });
    }

    res.status(200).json({
      _id: user._id,
      username: user.username,
      email: user.email,
      usertype: user.usertype,
      token: user._id.toString(),
    });
  } catch (error) {
    res.status(500).json({ msg: error.message });
  }
}
```

authRoutes.js

1. Login & register api

```
1 import express from "express";
2 import { registerUserController, loginUserController } from "../controllers/authController."
3
4 const router = express.Router();
5
6 router.post("/register", registerUserController);
7 router.post("/login", loginUserController);
8
9 export default router;
```

Frontend Development:

Core Tech

- **React JS**

Builds the user interface using reusable components (Navbar, Login, Register).

- **React Context API**

Manages global state (GeneralContext.jsx) to handle user sessions and app-wide data.

- **React Router**

Handles navigation between the Landing page and role-specific dashboards.

- **Role-Based Pages**

Organizes views into Admin, Client, and Freelancer folders for distinct user experiences.

- **Axios**

Connects the frontend components to your Node.js API routes for real-time

data flow.

Frontend Folder:

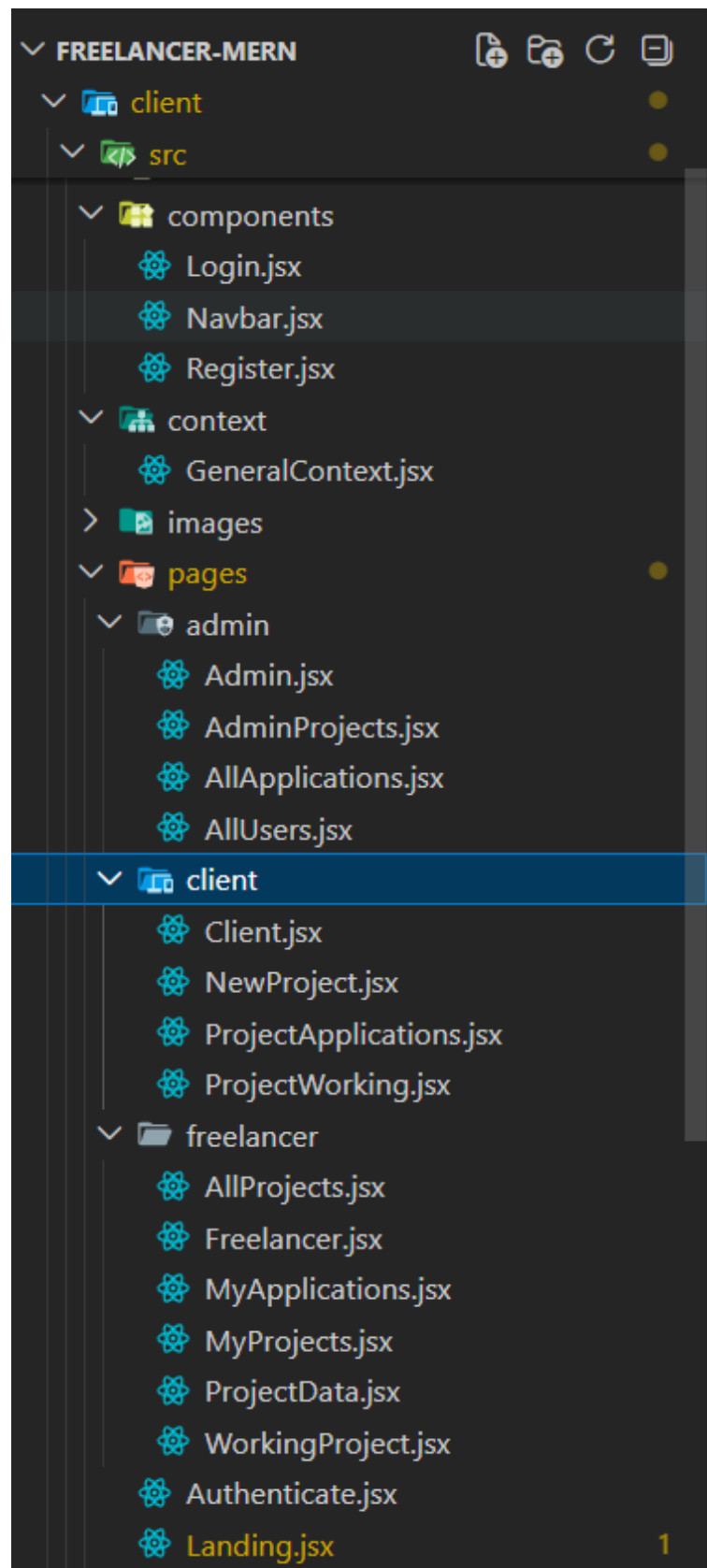


Figure of Frontend

Landing.jsx

1. Navbar

```
<nav className="navbar" flex justify-between items-center px-8 py-4">
  <span onClick={() => scrollTo(heroRef)}
    style={{ fontSize: 22, fontWeight: 800, color: '#2563eb', cursor: 'pointer', letterSpacing: '-0.5px' }}>
    SB Works
  </span>
  <div style={{ display: 'flex', alignItems: 'center', gap: 32 }}>
    <div style={{ display: 'flex', gap: 28, fontSize: 14, fontWeight: 600, color: '#475569' }}>
      {[ 'Home', 'Workflow', 'Contact' ].map((label, i) => (
        <button key={label}
          onClick={() => i === 0 ? scrollTo(heroRef) : i === 1 ? scrollTo(workflowRef) : scrollTo(contactRef)}
          style={{ background: 'none', border: 'none', cursor: 'pointer', fontWeight: 600,
            color: '#475569', fontSize: 14, transition: 'color 0.2s' }}
          onMouseEnter={e => e.target.style.color = '#2563eb'}
          onMouseLeave={e => e.target.style.color = '#475569'}>
            {label}
          </button>
        </div>
      </div>
      <div style={{ display: 'flex', gap: 10 }}>
        <button className="btn-ghost" onClick={() => navigate('/authenticate')}>Log In</button>
        <button className="btn-primary" style={{ padding: '8px 22px', fontSize: 14 }}
          onClick={() => navigate('/authenticate?type=register')}>Sign Up</button>
      </div>
    </div>
  </nav>
```

2. Hero Section:

```
<section ref={heroRef} className="hero-bg" style={{ display: 'flex', alignItems: 'center', padding: '0 5vw' }}>
  <div style={{ maxWidth: 1200, margin: '0 auto', width: '100%',
    display: 'grid', gridTemplateColumns: '1fr 1fr', gap: 60, alignItems: 'center', padding: '80px 0' }}>
    <div>
      <div className="au1 section-pill">Freelance Platform · India</div>
      <h1 className="au2" style={{ fontSize: 'clamp(38px, 5vw, 62px)', fontWeight: 800,
        lineHeight: 1.1, letterSpacing: '-1.5px', color: '#0f172a', margin: '0 0 20px' }}>
        Where Talent Meets{' '}
        <span className="grad-text">Real Opportunities</span>
      </h1>
      <p className="au3" style={{ fontSize: 18, color: '#64748b', lineHeight: 1.7,
        maxWidth: 420, margin: '0 0 36px' }}>
        SB Works connects serious freelancers with genuine clients.
        No spam. No fake projects. Just focused collaboration.
      </p>
      <div className="au4" style={{ display: 'flex', gap: 14, flexWrap: 'wrap', marginBottom: 36 }}>
        <button className="btn-primary" onClick={() => navigate('/authenticate?type=register')}>Start as Client</button>
        <button className="btn-outline" onClick={() => navigate('/authenticate?type=register')}>Start as Freelancer</button>
      </div>
      <div className="au4" style={{ display: 'flex', gap: 24, flexWrap: 'wrap', fontSize: 13,
        color: '#64748b', fontWeight: 600 }}>
        {[ '✓ Verified Projects', '✓ Zero Spam', '✓ Free to Join' ].map(t => (
          <span key={t} style={{ color: '#16a34a' }}>{t}</span>
        </div>
      </div>
      <div className="float"><HeroIllustration /></div>
    </div>
  </section>
```

3. Working:

```
<section ref={workflowRef}
  style={{ background: 'linear-gradient(135deg,#0f172a 0%,#1e1b4b 100%)', padding: '100px 5vw' }}>
  <div style={{ maxWidth: 1100, margin: '0 auto' }}>
    <div style={{ textAlign: 'center', marginBottom: 64 }}>
      <div className="section-pill" style={{ background: 'rgba(255,255,255,0.08)',
        borderColor: 'rgba(255,255,255,0.15)', color: '#93c5fd' }}>
        📋 How It Works
      </div>
      <h2 style={{ fontSize: 38, fontWeight: 800, color: 'white',
        letterSpacing: '-1px', margin: '0 0 16px' }}>
        From posting to <span className="grad-text">payment</span>
      </h2>
      <p style={{ color: '#94a3b8', fontSize: 16, maxWidth: 500, margin: '0 auto' }}>
        Three clear steps, zero confusion. The whole process is transparent and structured.
      </p>
    </div>

    <div style={{ display: 'grid', gridTemplateColumns: 'repeat(3,1fr)', gap: 32 }}>
      {
        [
          { n: '01', icon: '📋', title: 'Create & Explore',
            desc: 'Clients post detailed, verified projects. Freelancers browse and apply with focused proposals.',
            img: '/src/images/step1.svg' },
          { n: '02', icon: '💬', title: 'Connect & Assign',
            desc: 'Discuss scope, timelines and budget. Both sides agree before the project is officially assigned.',
            img: '/src/images/step2.svg' },
          { n: '03', icon: '💰', title: 'Deliver & Get Paid',
            desc: 'Submit work, get client approval, receive payment – all through a transparent workflow.',
            img: '/src/images/step3.svg' },
        ].map(({ n, icon, title, desc, img }) => (
          <div key={n} style={{ background: 'rgba(255,255,255,0.05)', borderRadius: 20,
            border: '1px solid rgba(255,255,255,0.1)', padding: '36px 28px',
            transition: 'all 0.3s', cursor: 'default' }}
            onMouseEnter={e => {
              e.currentTarget.style.background = 'rgba(255,255,255,0.09)'
              e.currentTarget.style.transform = 'translateY(-6px)'
            }}
            onMouseLeave={e => {
              e.currentTarget.style.background = 'rgba(255,255,255,0.05)'
              e.currentTarget.style.transform = 'translateY(0)'
            }}
          >
            <div style={{ fontSize: 11, fontWeight: 800, color: '#6366f1', letterSpacing: '0.1em',
              marginBottom: 16 }}>STEP {n}</div>
            <img src={img} alt={title} style={{ height: 80, marginBottom: 20 }}
              onError={e => { e.target.style.display = 'none' }} />
            <h3 style={{ fontSize: 20, fontWeight: 700, color: 'white', marginBottom: 12 }}>
              {icon} {title}
            </h3>
            <p style={{ fontSize: 14, color: '#94a3b8', lineHeight: 1.7 }}>{desc}</p>
            <div style={{ marginTop: 24, width: 40, height: 40, borderRadius: '50%',
              background: 'linear-gradient(135deg,#2563eb,#7c3aed)',
              display: 'flex', alignItems: 'center', justifyContent: 'center',
              color: 'white', fontWeight: 800, fontSize: 15,
              boxShadow: '0 4px 16px rgba(99,102,241,0.4)' }}>
              {n.replace('0', '')}
            </div>
          </div>
        ))
      }
    </div>
  </div>
</section>
```

```
    <div style={{ display: 'grid', gridTemplateColumns: 'repeat(3,1fr)', gap: 32 }}>
      {
        [
          { n: '01', icon: '📋', title: 'Create & Explore',
            desc: 'Clients post detailed, verified projects. Freelancers browse and apply with focused proposals.',
            img: '/src/images/step1.svg' },
          { n: '02', icon: '💬', title: 'Connect & Assign',
            desc: 'Discuss scope, timelines and budget. Both sides agree before the project is officially assigned.',
            img: '/src/images/step2.svg' },
          { n: '03', icon: '💰', title: 'Deliver & Get Paid',
            desc: 'Submit work, get client approval, receive payment – all through a transparent workflow.',
            img: '/src/images/step3.svg' },
        ].map(({ n, icon, title, desc, img }) => (
          <div key={n} style={{ background: 'rgba(255,255,255,0.05)', borderRadius: 20,
            border: '1px solid rgba(255,255,255,0.1)', padding: '36px 28px',
            transition: 'all 0.3s', cursor: 'default' }}
            onMouseEnter={e => {
              e.currentTarget.style.background = 'rgba(255,255,255,0.09)'
              e.currentTarget.style.transform = 'translateY(-6px)'
            }}
            onMouseLeave={e => {
              e.currentTarget.style.background = 'rgba(255,255,255,0.05)'
              e.currentTarget.style.transform = 'translateY(0)'
            }}
          >
            <div style={{ fontSize: 11, fontWeight: 800, color: '#6366f1', letterSpacing: '0.1em',
              marginBottom: 16 }}>STEP {n}</div>
            <img src={img} alt={title} style={{ height: 80, marginBottom: 20 }}
              onError={e => { e.target.style.display = 'none' }} />
            <h3 style={{ fontSize: 20, fontWeight: 700, color: 'white', marginBottom: 12 }}>
              {icon} {title}
            </h3>
            <p style={{ fontSize: 14, color: '#94a3b8', lineHeight: 1.7 }}>{desc}</p>
            <div style={{ marginTop: 24, width: 40, height: 40, borderRadius: '50%',
              background: 'linear-gradient(135deg,#2563eb,#7c3aed)',
              display: 'flex', alignItems: 'center', justifyContent: 'center',
              color: 'white', fontWeight: 800, fontSize: 15,
              boxShadow: '0 4px 16px rgba(99,102,241,0.4)' }}>
              {n.replace('0', '')}
            </div>
          </div>
        ))
      }
    </div>
  </div>
</section>
```

Project Overview:

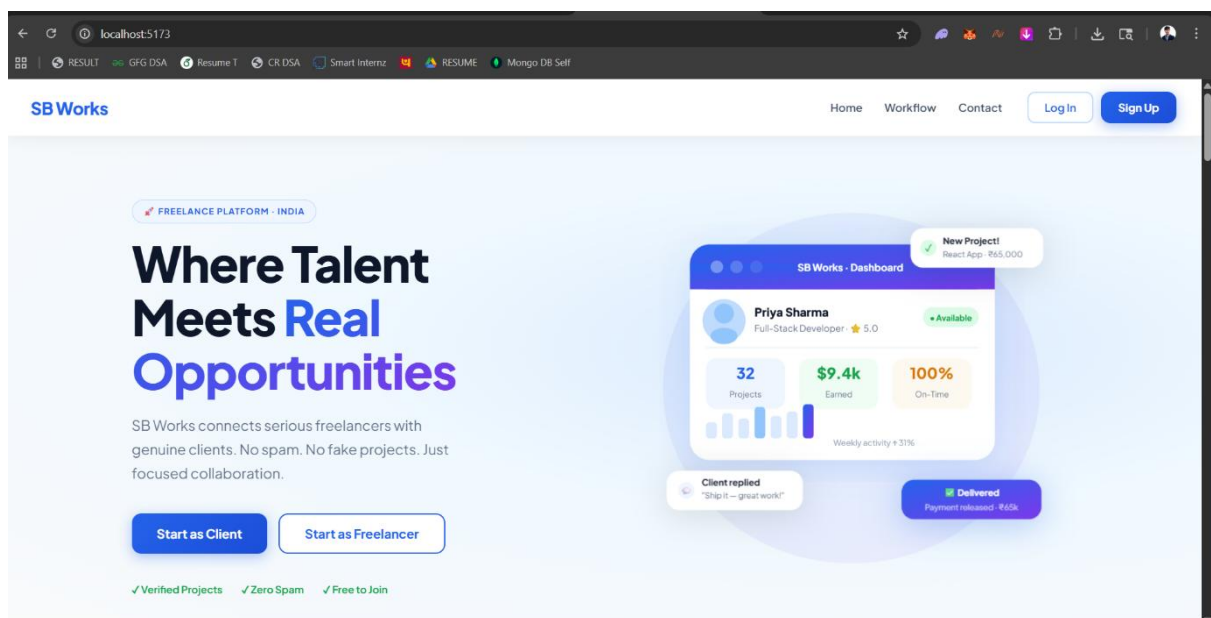
1. **Landing Page:** A **landing page** is the **first page users see** when they visit your website.

Purpose:

- Attract users
- Explain what your platform does
- Convert visitors into users (signup/login)

Key Sections:

- Hero section (headline + tagline)
- Call-to-action (CTA) buttons (e.g., “Get Started”, “Join Now”)
- Features or benefits
- Testimonials (optional)
- Footer



2. **Login Page:** A **login page** allows **existing users** to access their account.

Purpose:

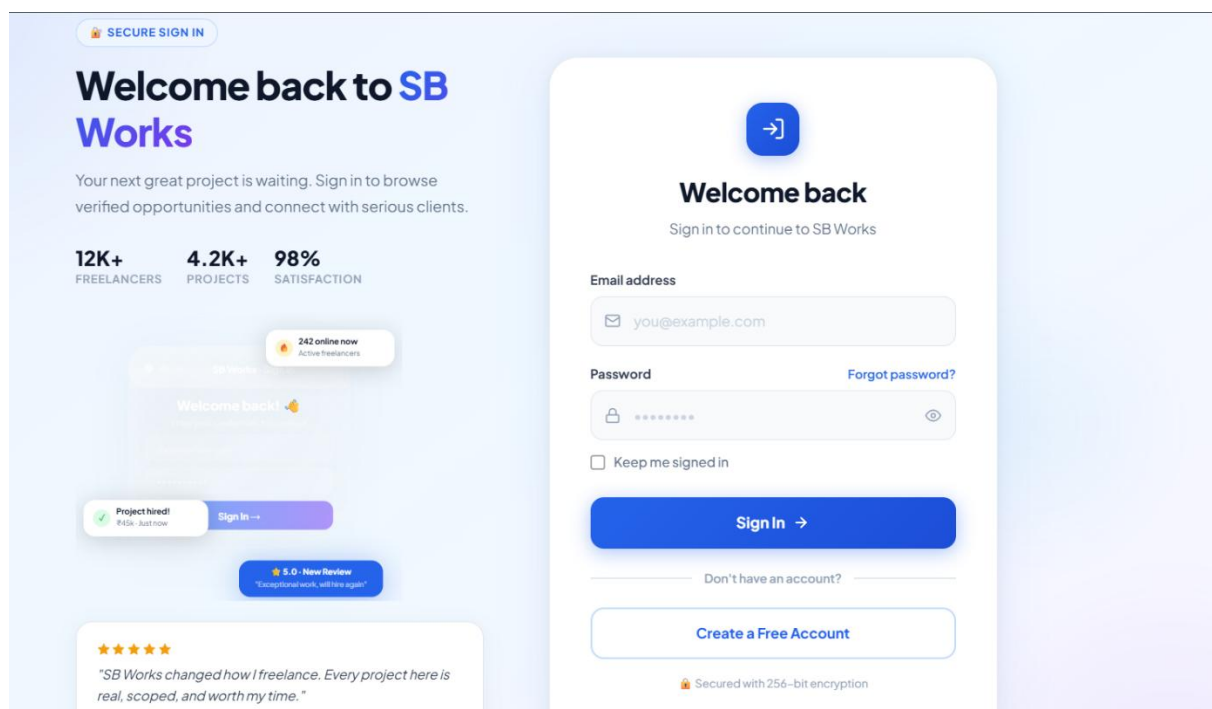
- Authenticate users
- Provide secure access

Fields:

- Email / Username
- Password
- “Forgot Password” option
- Login button

Function:

- Checks credentials → If correct → Redirect to dashboard



3. Register Page: A **register page** is used to **create a new account**.

Purpose:

- Collect user information

- Store in database

Fields:

- Name
- Email
- Password
- Confirm Password
- Role (Freelancer / Client)

Function:

- Creates account → Saves data → Redirects to login

✓ Verified Projects ✓ Secure Payments ✓ Free to Join

Create your account

Join thousands of freelancers and clients

Username

 yourname

Email address

 you@example.com

Password

 Min. 8 characters 

I am a...



Freelancer
Find projects



Client
Hire talent

Create Account →

Already have an account?

Sign In Instead

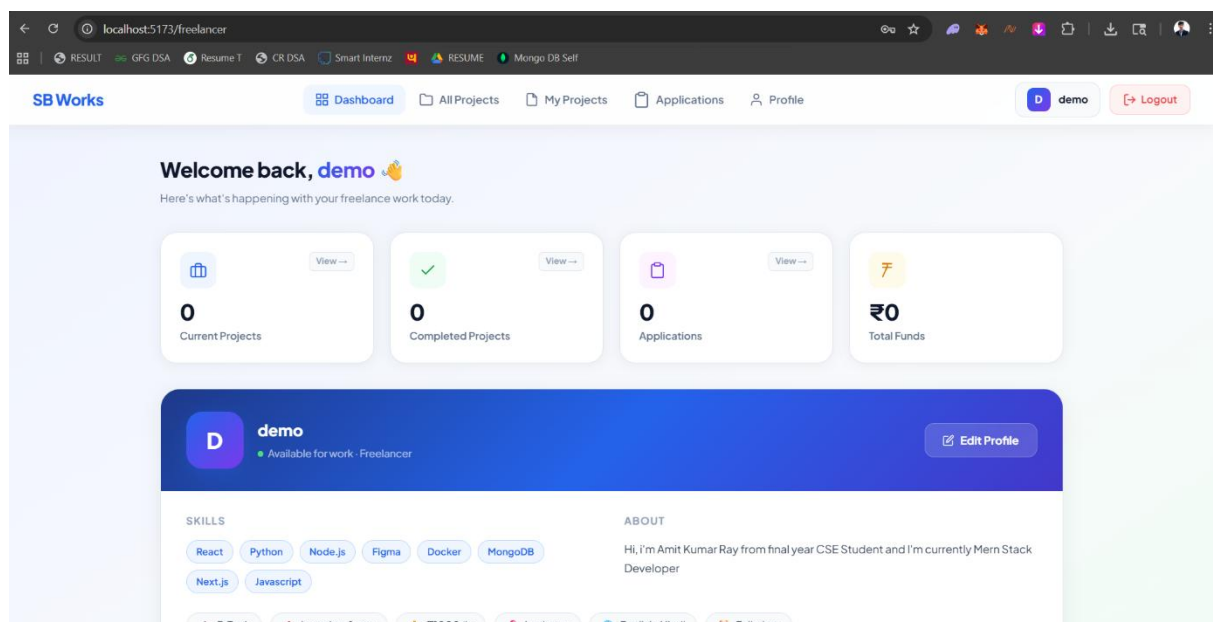
4. **Freelancer Page:** This is the **dashboard for freelancers** after login.

Purpose:

- Help freelancers manage their work

Features:

- View available jobs
- Submit proposals (bids)
- Track earnings
- Manage profile
- View messages from clients



5. **Client Home Page:** This is the **dashboard for clients (job posters)**.

Purpose:

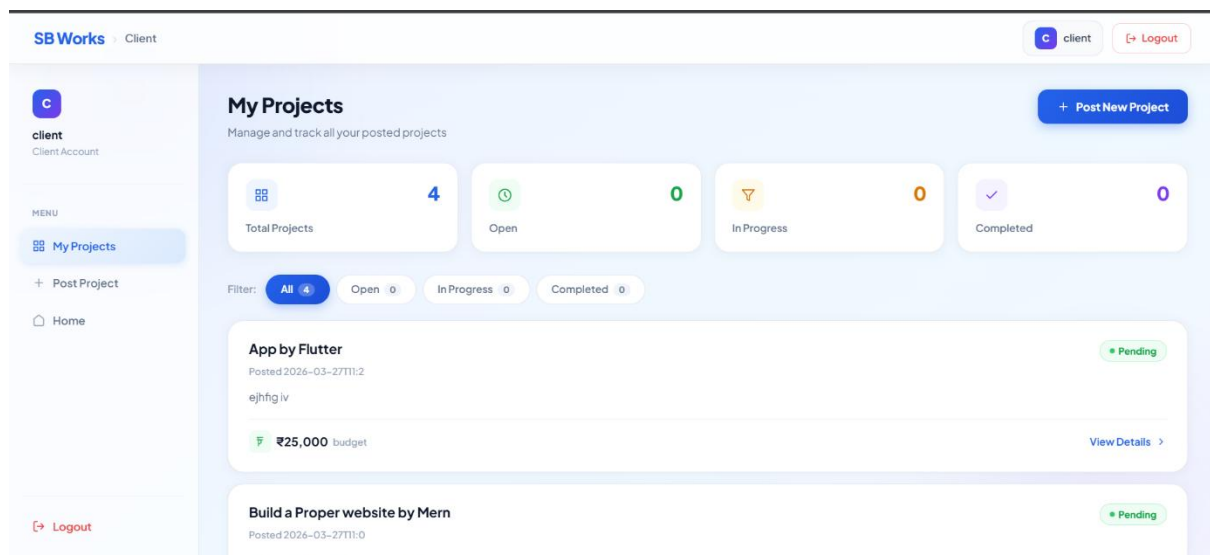
- Manage projects and freelancers

Features:

- Post a job
- View proposals from freelancers
- Hire freelancers
- Track project progress
- Payment management

Example:

- Client posts a job → receives bids → selects freelancer



6. Admin Home Page: The admin portal is for the system administrator.

Purpose:

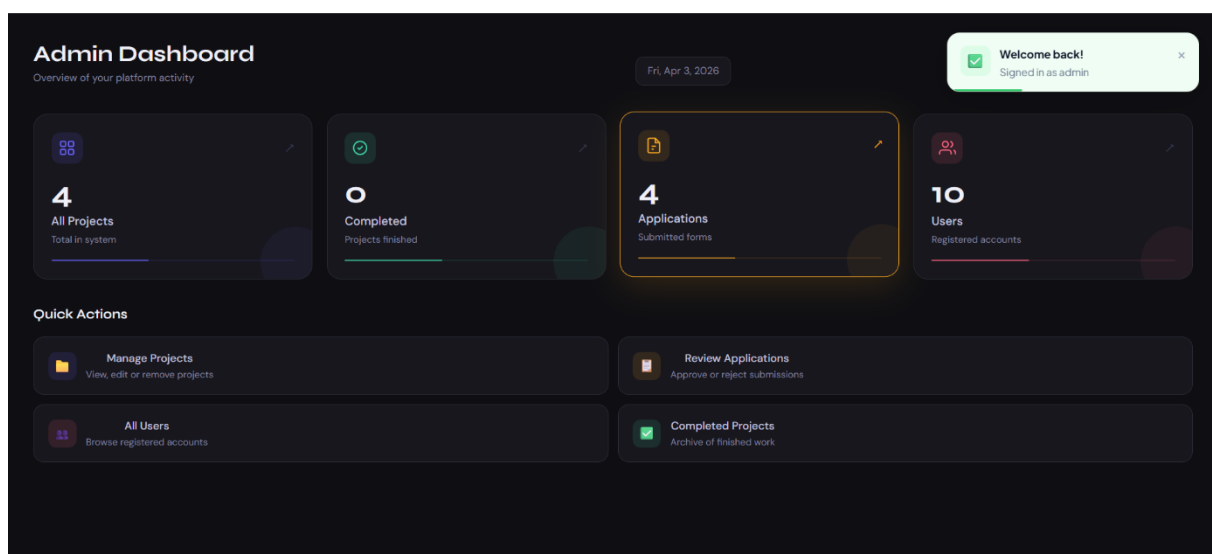
- Control and monitor the entire platform

Features:

- Manage users (freelancers & clients)
- Approve/reject accounts or jobs
- View platform analytics
- Handle reports & complaints
- Manage payments or system settings

Example:

- Admin can ban a user or remove fake job posts



Project Execution:

- **Step for Project Execution**

Step 1: Download and Open the Project

1. First, clone or download the project folder (named **freelancer-mern**) from the repository or your local source.
2. Once downloaded, open the project folder in VS Code or any code editor of your choice.

- **You will see two main subfolders:**

client → contains the frontend (React) code

server → contains the backend (Node.js + Express) code

Step 2: Set Up the Frontend (React App):

- a. Open a terminal and navigate into the client folder:

```
cd client
```

- b. Install all the required frontend packages:

```
npm install
```

- c. Once installation is done, start the React development server:

```
npm run dev
```

- d. The app should now be running on:

<http://localhost:5173>

Step 3: Set Up the Backend (Express Server):

- a) Open a new terminal tab/window or split the terminal.
- b) Navigate into the server folder:

```
cd ../server
```

- c) Install all backend dependencies (like Express, Mongoose, etc.):

```
npm install
```

Step 4: Configure Environment Variables:

- a) Inside the **server** folder, create a new file named **.env** (no file extension).
- b) In that **.env** file, add your MongoDB connection string:
MONGO_URI=
mongodb+srv://freelance:password@cluster0.rtczxo.mongodb.net
/freelancerDB?retryWrites=true&w=majority

Step 5: Start the Backend Server:

- a) Inside the same server folder, run the backend server using:

```
nodemon index.js / npm run dev
```

- b) The server should start on:

<http://localhost:8000>

Demo Video Link:

https://drive.google.com/file/d/1cPKtZjJjYE5FJ9uA1ITxS3_L6s91ZPxr/view?usp=sharing

